

Keeping a Passed-Through USB Storage Device Alive

across Same-Host VM Live Migration with a Multi-Client vfio-user Backend

Anonymous Submission

Paper ID: `usbvfioid-usb-migration`

Author names, affiliations and contact addresses are withheld for double-blind review.

Abstract—Passing a physical USB device through to a virtual machine (VM) is attractive for storage, dongles and instrumentation, but it conflicts with live migration: the host-side device session (open file descriptors, claimed interfaces, endpoint state and the device’s internal state) cannot be serialised, so any migration path that tears the session down forces the guest to re-enumerate. This paper studies the concrete case of Cloud Hypervisor together with `usbvfioid`, a Rust vfio-user server that emulates an eXtensible Host Controller Interface (xHCI) controller. We first show, by measurement, that migrating a VM with a vfio-user device *deadlocks*: the destination blocks for 199.9s inside the protocol version handshake because the vfio-user server accepts exactly one connection, which the source still holds. We then present a design that removes the deadlock without modifying virtual machine monitor (VMM) source code: because the vfio-user handshake commands never touch the device backend, a wrapper that takes the backend lock *per command* rather than per connection lets a second client connect while the first client is still connected. A separate one-line bug in the protocol crate (an inverted reset-capability test) does require rebuilding the VMM against the fixed crate, because the crate is compiled into it. We identify and fix two defects that the hand-over exposes—a departing client tearing down the surviving client’s interrupt line and DMA mappings, and completion interrupts lost inside the hand-over window—and we describe a measurement harness that derives the verdict from the guest’s own log rather than from host-side observation. On a real 32GB exFAT stick passed through to an Ubuntu 22.04 guest, a 128MiB file copy that spans the migration completes in 20 out of 20 runs with a measured downtime of 4–21ms, byte-identical data and, at the USB level, no re-enumeration, no reset and no I/O error in the guest. A no-migration control, a release-build arm, a kick-disabled negative control and a naive detach/re-attach baseline separate the fixes from the conditions, and a fault-injection suite that enlarges the hand-over window on purpose shows that both hand-over fixes are load-bearing rather than incidental.

Index Terms—virtualisation, live migration, device passthrough, vfio-user, USB, xHCI, Cloud Hypervisor, KVM

I. INTRODUCTION

Virtual machines are routinely equipped with physical devices through passthrough so that they can use hardware that is either impossible or unattractive to emulate. Live migration—moving a running VM between VMM instances with a downtime short enough to be unnoticeable—is the

standard mechanism for host maintenance, load balancing and upgrades [1]. The two techniques, however, interact badly.

Memory and CPU state migrate well, and in-kernel VFIO devices can migrate when they implement the VFIO device migration interface [2]. USB devices are harder, because the host-side session is not part of the guest state. A guest talks to a USB device through a host controller; a passthrough implementation either emulates that controller and forwards individual transfers, or hands the whole controller to the guest. In the first case the host process that owns the physical device holds a session that is *not* part of the guest state: an open file descriptor, a claimed interface, endpoint configuration and, above all, the device’s own internal state. None of this can be serialised. Any migration that recreates the device on the destination therefore forces the guest to observe a disconnect and a re-enumeration, which at minimum interrupts I/O and can invalidate host-side state such as a mounted filesystem or an encrypted volume.

Cloud Hypervisor (CH) supports USB passthrough through the vfio-user protocol via `--user-device socket=...` [3]. The server side can be `usbvfioid` [4], which emulates an xHCI controller and forwards transfers to the physical device with `nusb` [5]. CH documents live migration [6], but the vfio-user path was never exercised under migration.

This paper starts from a measurement: migrating a CH VM that has a vfio-user device attached does not fail cleanly—it *hangs*, and it hangs on the destination VMM (Section III). We then design and implement a solution that makes a same-host migration of such a VM work, and we measure it end to end with real hardware.

A. Contributions

- 1) We characterise, by measurement and source inspection, why a vfio-user device deadlocks a CH live migration: the destination VMM blocks in the version handshake because the server accepts a single connection that the source still holds. We quantify it (199.9s of zero progress; the destination never gets a reply) and show that even a successful takeover would not transfer device state (Section III).

- 2) We present a *multi-client* vfio-user backend design that removes the deadlock without modifying the VMM. The key observation is that the handshake commands never reach the device backend, so the backend lock can be taken per command instead of per connection (Section IV).
- 3) We identify and fix two defects that only appear once a hand-over is possible—a departing client disabling the surviving client’s interrupt line and unmapping its DMA regions, and completion interrupts being delivered to the departing VMM’s interrupt descriptor inside the hand-over window (Sections IV-C and IV-D)—and a reset-capability parsing bug in the protocol crate that made every connection reset the device (Section V-B).
- 4) We build a reproducible end-to-end harness, including a guest image and a measurement method whose verdict is read from the *guest’s own log* rather than from host-side observation, and we report the design decisions that this method forced us to change (Section V).
- 5) We evaluate the result with a real USB stick: 20 out of 20 migrations of a VM that is copying a 128 MiB file at the time of the migration succeed with 4–21 ms downtime, byte-identical data and no re-enumeration or reset in the guest, and we separate the fixes from luck with a no-migration control, a release-build arm, a kick-disabled negative control, a naive detach/re-attach baseline, and a fault-injection suite that makes the hand-over race happen by construction (Sections VI–VI-H).

B. Paper organisation

Section II reviews live migration, device passthrough and USB passthrough alternatives. Section III analyses the measured deadlock. Section IV presents the design, Section V the implementation. Section VI evaluates, Section VII discusses limitations, and Section VIII concludes.

II. BACKGROUND AND RELATED WORK

A. Live migration

Live migration of VMs was popularised by Clark *et al.* [1], who introduced pre-copy migration: pages are copied while the VM runs, and a final stop-and-copy phase transfers the remaining dirty pages while the VM is paused. KVM [7] made this a mainstream kernel mechanism, and modern VMMs such as Firecracker [8] and Cloud Hypervisor [9] implement variants of it. Cloud Hypervisor documents both TCP and UNIX-socket transports; for same-host migration it offers `memory_mode=memfds`, which passes the guest memory *backing file descriptors* (`memfd`, [10]) over the socket so that source and destination map the same memory instead of copying it [6]. This is the mode we use: it makes the memory identical in both VMMs, which is what allows a vfio-user server on the host to keep its DMA mappings.

B. Device passthrough: VFIO and vfio-user

VFIO is the Linux framework that exposes a device to user space with an IOMMU-backed DMA model [11]. In

the kernel-VFIO model the VMM maps device BARs and registers event descriptors directly. vfio-user moves the same programming model to a user-space server reached over a UNIX socket [12]: a client (the VMM) queries regions, reads and writes device memory, maps guest memory for device DMA, registers interrupt file descriptors, and may reset the device. The protocol specification is maintained with QEMU [12] and reference implementations exist in `libvfio-user` [13]. Cloud Hypervisor implements the client side [3]; the `vfio_user` crate [14], part of the `rust-vmm` collection [15], implements both sides and is used by CH and by `usbvfioid`.

Two lines of work are directly adjacent to ours. A QEMU patch series proposes protocol-level live migration for vfio-user devices by reusing the generic VFIO v2 migration infrastructure—migration data read/write and DMA logging for dirty-page tracking [16]. That is the right long-term answer, and it is *not* what this paper does: it transfers device state, which requires both ends to run a compatible server and does not preserve a host-side session that cannot be expressed as state. Our problem is the same-host case, where the server process can simply stay alive and only the VMM that talks to it changes; what we contribute there is the removal of the connection-level serialisation that otherwise makes the hand-over impossible, plus the hand-over correctness rules that a state-transfer design would also need. The second line is earlier work on migrating USB devices between hosts through a proxy mechanism [17], which likewise assumes a medium that carries the device rather than preserving a local session.

C. USB passthrough alternatives

Table I compares the options we considered. Passing through with a user-space server over vfio-user is the approach of `usbvfioid` and the subject of this paper: it keeps the guest driver unmodified and confines the device to an unprivileged process, at the cost of an emulated controller in the transfer path. USB/IP [18], [19] tunnels USB traffic over a network and is the natural answer for *cross-host* device sharing, but the device remains attached to the source host and depends on an external transport. QEMU also provides USB to a guest, either by emulating devices or by attaching a physical one with `usb-host` [20]. Libvirt does permit a domain with a USB host device to migrate [21]—the device is detached on the source and re-attached on the destination, which libvirt’s own comment calls “migrated” in quotation marks—but the host-side session is not preserved: the guest sees the device disappear and come back, which is exactly the naive alternative we measure in Section VI-D. USB redirection sends the traffic to a remote client [22] rather than preserving a host-side session.

D. Emulated xHCI and USB mass storage

`usbvfioid` presents an xHCI controller [23] to the guest. The Linux xHCI driver discovers the controller, performs a host controller reset, enumerates the attached devices and then drives transfers through command, event and transfer rings in guest memory. Mass-storage devices use the Bulk-Only

TABLE I
USB PASSTHROUGH APPROACHES.

Approach	Where device state lives	Consequence for migration
Host <code>usbfs</code> process with an emulated <code>xHCI</code> in the guest [4] USB/IP [18], [19]	host process, not serialisable source host, exported over a network	session must survive; used here device stays on the source host; transport in the data path
QEMU <code>usb-host</code> (physical passthrough) [20], [21]	device bound inside the VMM process	migration detaches and re-attaches the device; the guest sees it go away (Section VI-D)
USB redirection (<code>usbredir</code>) [22]	device stays on the host, traffic goes to a remote client	the client reconnects the transport; no host session is preserved
Kernel VFIO migration (non-USB) [2]	device-defined migration state	requires device support; not applicable to USB

Transport protocol [24], which the kernel’s `usb-storage` module implements. Our measurements use this path because it is the most common passthrough workload and because a bulk transfer makes the consequences of a lost completion immediately visible as an I/O error.

Figure 1 summarises the resulting architecture. The physical device is owned by a host process for the entire lifetime of the VM, including the migration; the VMM is only a client of that process, and the guest sees an ordinary controller.

III. PROBLEM ANALYSIS

A. Experimental setup

All measurements were taken on one bare-metal host running Debian 13 with Linux 6.17.4 (Proxmox kernel), 8 cores and 62 GiB of RAM, which runs the Kernel-based Virtual Machine (KVM). The host was *not* idle: four unrelated KVM virtual machines (16, 8, 2 and 1 GiB of RAM) were running throughout, and no CPU pinning was used. We report this because it is an uncontrolled load confounder and, as Section VI-B notes, the most likely explanation for the spread in the measured downtime. Table II lists the software under test. The guest has 2 GiB of RAM and one vCPU, runs Ubuntu 22.04.4 (kernel 5.15.0-94-generic) and is booted directly by CH. The physical device is a 32 GB USB 2.0 High-Speed stick (Innoster 1f75:0903, exFAT data partition) attached at `/dev/bus/usb/001/007`. The server binary under test was an unoptimised cargo debug build, run with verbose logging (`-v`) and USB packet capture enabled; Section VI-K discusses the resulting measurement overhead.

B. Measured baseline: the migration deadlocks

Figure 2 illustrates what we measured. A source VM with `--user-device socket=...` is migrated with `send-migration` using `destination_url=unix:...` and

TABLE II
SOFTWARE UNDER TEST.

Component	Version
Cloud Hypervisor	v53.0-520-gc24527002 (crate version 54.0.0) [9]
<code>usbvfioid</code>	baseline 4d2c5af [4]
<code>vfio_user</code> crate	0.1.5 [14]
<code>nusb</code> crate	0.2.7 [5]
Guest	Ubuntu 22.04.4, kernel 5.15.0-94-generic
Device	32 GB exFAT USB stick, USB 2.0 High-Speed

`memory_mode=memfds`. The destination VMM starts from the migrated configuration and reaches the point where it restores the `vfio-user` device:

```
device_manager.rs:4749 Restoring virtio-pci
_vfio_user0 resources
```

and then makes no further progress. We waited 199.9 s; there was no additional log output, no error, and the destination VMM’s control API stopped answering. The source VM remained paused in the migrating state. This measurement predates the fixes and comes from the standalone phase-0 experiment recorded in the project’s development log (the round-3 batches of Section VI measure the fixed system and do not contain it). Three observations pin the cause down: the server logged exactly *one* client version handshake (the source’s), a socket inspection showed `Recv-Q=1` on the listening socket, i.e. the destination’s connection was established but never accepted, and the destination had blocked inside

```
pci/src/vfio_user.rs:4338 →
vfio_user::Client::new()
                                →
negotiate_version() → read_exact()
```

waiting for a reply that never comes.

C. Root cause: one connection, one accept

The server side of `vfio_user` 0.1.5 serves a connection by calling `listener.accept()` once and then looping over commands until the peer closes. A second client therefore *connects successfully* (the kernel completes the handshake into the listen backlog) but is never served, which is exactly the “hangs forever” symptom above. A cross-host migration would have the same problem, and additionally the destination would have no device: the guard for that exists only for kernel-VFIO devices (VFIO device does not support migration), not for `vfio-user`.

D. Device state would not be transferred anyway

Even with the hand-over solved, the `vfio-user` device carries almost no migration state in this version. Its `Pausable`, `Transportable` and `Migratable` implementations are empty, and its snapshot contains only the generic PCI state (configuration space, `INTx/MSI/MSI-X` configuration and the `MSI-X` table); device BAR state, opaque device state and DMA dirty tracking are absent. There is no device-state region in the protocol implementation and no parameter with which the destination could be pointed at a different `vfio-user` server. A protocol-level migration mechanism has been proposed for `vfio-user` [16], but the implementation used here carries no

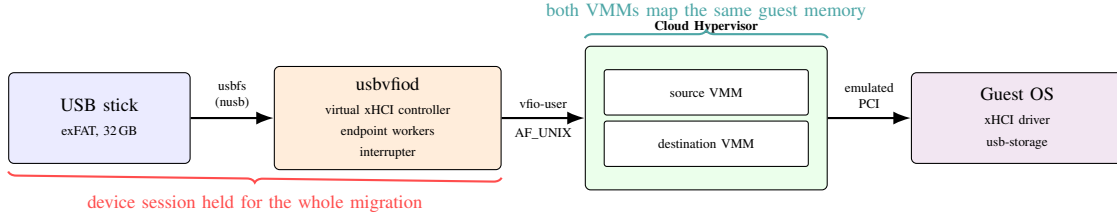


Fig. 1. System architecture. `usbvfiod` owns the physical USB device and emulates an xHCI controller; the VMM reaches it over `vfiio-user`. During a same-host migration the server process is never restarted, so the device session and the controller state survive, and both VMMs map the same guest memory.

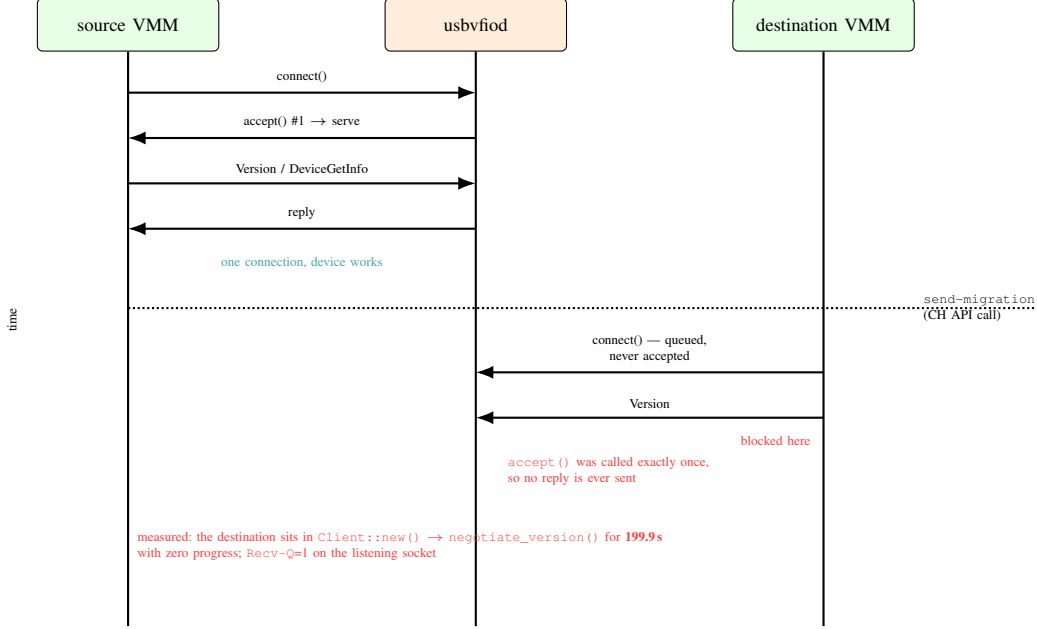


Fig. 2. Measured deadlock of the unmodified stack. The `vfiio-user` server calls `accept()` exactly once, so the destination's connection is queued and its version handshake is never answered; the migration never completes.

device state. This is why our design deliberately does *not* try to move device state: for a same-host migration the server process simply stays alive and keeps the session, and only the *client* that the server talks to changes.

IV. DESIGN

A. Goals and non-goals

The goal is that a same-host live migration of a VM with a passed-through USB storage device completes, and that the guest does not observe a device change. Concretely we require: (i) the migration completes; (ii) the guest sees no re-enumeration and no reset; (iii) data transferred across the migration is neither corrupted nor lost (duplication is not tested, as Section VI-K explains); (iv) the VMM needs no source change, being only rebuilt against the fixed protocol crate. Non-goals are cross-host migration, USB classes other than bulk storage, and isochronous transfers.

B. Multi-client backend with a per-command lock

The deadlock exists because the server serialises *connections* even though the commands it handles are

mostly independent of the device. In the crate, only `RegionRead`, `RegionWrite`, `DmaMap`, `DmaUnmap`, `SetIrqs` and `DeviceReset` invoke the backend; `Version`, `DeviceGetInfo`, `DeviceGetRegionInfo` and `GetIrqInfo` do not. Consequently a second client can complete its handshake while the first is still attached, provided that the backend is shared with a lock taken *per command*.

Figure 3 shows the resulting structure. A shared state object owns the backend and the interrupt-ownership bookkeeping, and hands out one handle per connection. Each handle implements the server backend trait by delegating to the shared backend under a mutex for the duration of a single call. The `accept` loop is replicated over a configurable number of threads, so the process can serve the source and the destination simultaneously. The default keeps the historical single-client behaviour unchanged, which matters for the systemd integration where the server is expected to exit when its only client goes away.

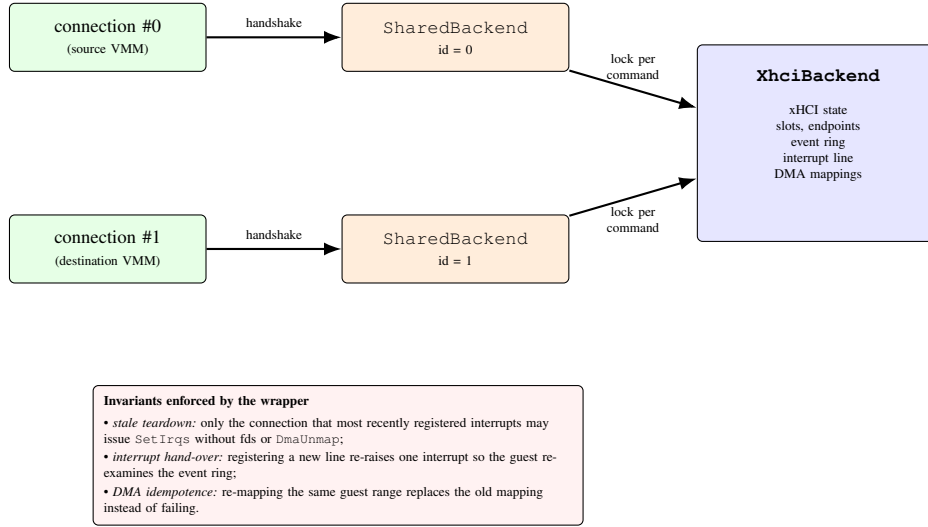


Fig. 3. Multi-client backend. Handshake commands never take the lock, so the destination VMM can connect while the source is still attached; the device state itself lives in the shared backend and therefore survives the hand-over.

C. Ownership: a departing client must not tear the device down

Allowing two clients exposes a hazard that a single-client server never had. When the migration completes, the *source* VMM shuts its device down and sends two destructive commands: `SetIrqs` with an empty descriptor list (disable interrupts) and `DmaUnmap` for the regions it mapped. Both arrive *after* the destination has already registered its own interrupts and DMA mappings, and both would be applied to the shared backend. The result we measured is severe: the destination is left with a dummy interrupt line, no completion is ever signalled, the guest’s commands time out and the running I/O fails. Section VI-H reproduces exactly this by disabling the guard on purpose: five runs out of five fail, with a 36 s command timeout and repeated I/O errors in the guest’s kernel log.

The wrapper therefore tracks which connection most recently registered interrupts and lets only that connection issue the destructive counterparts. Teardown from any other connection is logged and ignored. In every run we observed the interrupt-disable half of this guard fire (the server logs the ignore); we did not observe a `DmaUnmap` reach the server on this path, so that half is implemented but not exercised by our workload. This is deliberately coarse: it is correct for the hand-over case, which is the case that needs it, and it cannot be triggered accidentally by the active client.

D. Interrupt hand-over correctness

A subtler, timing-dependent defect remains even after Section IV-C. The interrupter signals every event by writing to the currently installed interrupt line. During a hand-over there is a window in which a transfer may complete *before* the destination’s line is installed: the completion event is written into the guest’s event ring (which is in guest memory and therefore already correct), but the interrupt is delivered to

the event descriptor of the VMM that is exiting. If that completion is the last one outstanding, the guest never learns that it completed—it does not re-read its event ring without an interrupt—and the copy stops at a fixed byte count until the block layer’s command timeout expires; in our failing runs that produced repeated I/O errors rather than a recovery. If more commands were still queued, the next completion interrupts the guest on the new line and the guest drains the ring, including the event it was not told about. That is why the defect is intermittent, and why it is established in this paper by injection rather than by repeating natural runs (Section VI-H).

Our fix is to re-raise one interrupt immediately after a new line is installed. A spurious MSI-X interrupt is harmless: the guest inspects the event ring, finds the pending completion (or nothing) and continues. This converts a lost interrupt into at most one redundant ring scan, and the 5 s-window experiment shows that one such interrupt is sufficient even when the window is three orders of magnitude wider than the natural one.

E. DMA mapping idempotence

Because source and destination map the *same* guest memory under `memory_mode=memfds`, the destination re-issues `DmaMap` for ranges that the source already mapped. A naive “insert only” bus rejects the duplicate as an overlap. We make the DMA bus replace a mapping that starts at the same address, while still rejecting a genuine overlap at a different address, and we make a failed insert leave the bus untouched. We also implement the unmap path, which the VMM invokes when it destroys a device and which was previously unimplemented.

V. IMPLEMENTATION

A. *usbyfiod*

The multi-client change is confined to six files (381 added and 28 removed lines relative to the upstream revision): a

new `shared_backend` module implementing the wrapper and the ownership rules; the accept-loop and lifecycle logic in `main.rs`; a `--max-clients` option; the idempotent DMA bus; and the DMA, reset and interrupt-registration paths of the xHCI backend. Two further changes are small but load-bearing: moving the hand-over kick into the interrupter worker, and the two debug-only injection hooks used in Section VI-H. With those included the working tree differs from the upstream revision by seven files (516 added and 104 removed lines counting the one-line change to the USB backend, 515/103 excluding it). The device emulation itself is untouched. Unit tests grew by three (108 to 111 test attributes; `cargo test` reports 112 passing), and the project’s strict lint configuration (`clippy` with all, nursery and cargo groups denied) passes.

B. Protocol crate

One behavioural bug in `vfio-user` 0.1.5 had to be fixed for the reset semantics to be sane. The client parses the device’s reset capability by comparing the masked flag word with *inequality*, which inverts the result: a server that advertises “not resettable” is reported as resettable, so the VMM issues a device reset on every connection, including on the destination’s connection during a migration. The fix is a one-line equality comparison. Because the crate is consumed by the VMM, the fix is applied to the VMM’s dependency graph rather than to `usbvfiod`, which only uses the server side.

We should be explicit about what this fix is and is not supported by. It is established by inspection of the published crate and by the behaviour of the unpatched build, in which CH’s destination-side device creation fails and the experiment cannot run at all; that is why every measurement in this paper uses the patched build. It is *not* isolated by an ablation arm: no log in the round-3 batches contains a `DeviceReset` command, because with the fix in place the reset is not offered and therefore never issued. A controlled comparison would require an arm that patches the parse to force a reset, which we did not run, so the claim we make is the narrow one—the parse is inverted and the build with it corrected is necessary to obtain the results here—rather than a measured statement about how much the reset alone costs.

C. Cloud Hypervisor integration

No CH source file was modified, but the result does require a *custom CH build*: the integration artefact is a `[patch.crates-io]` entry that points CH at the fixed crate, and reproduction therefore needs that build. Because the crate lives in a monorepo whose manifest also references a sibling `vfio-bindings` crate that has diverged from the published version, the patch deliberately uses a variant of the fix branch in which that crate resolves from `crates.io`; otherwise two copies of `vfio-bindings` coexist and their types no longer unify. This is a build-level detail, not a design change, and it is documented so that it can be dropped once the fix is released upstream.

D. Guest image and measurement harness

Measurement honesty turned out to require as much care as the device code, and we record the decisions because they changed our verdict twice.

Guest image. The Ubuntu live-server ISO cannot be booted directly: its `initrd` insists on finding a live medium and ignores `root=`, and the minimal `squashfs` ships an empty `/lib/modules`, so `usb-storage` cannot be loaded after the switch. We therefore unpack the ISO, graft the module tree out of the `initrd`, rebuild the `initrd` without the casper hooks and pack a bootable `ext4` image. The `exFAT` driver, which real sticks use, lives in `linux-modules-extra` and is merged in explicitly.

Console capture. CH opens a serial `file=` target with `File::create()`, so the destination VMM truncates the log when it starts; the pre-migration evidence disappears exactly when it becomes interesting. We capture the console through a FIFO kept open by a reader that never sees EOF.

Verdict source. The migration also re-creates the destination’s serial device, which resets the guest TTY. Progress written only to the console then looks like a hang although the guest is fine. We therefore make the guest log to a file and treat the serial console as a best-effort mirror, and we compute the verdict from the guest’s own log after stopping the VMs and mounting the disk image—with the journal replayed, because a read-only `noload` mount hides the most recent writes. A guest heartbeat service, finally, distinguishes “guest frozen” from “console broken”, and reports the size of the in-progress copy so the harness can place the migration at a fixed fraction of the transfer rather than after a fixed wall-clock lead.

Fault injection. Two hooks in the server exist only in debug builds and only take effect when an environment variable is set, so an ordinary run cannot reach them: one sleeps for a configured number of milliseconds inside the `SetIrqs` handler *before* the new interrupt line reaches the interrupter worker, and the other makes every connection believe it owns the device, which reintroduces the stale-teardown defect. They are what turns the hand-over window from a race that happens by chance into one that happens by construction; Section VI-H uses them, and the first row of that section’s table checks that they are inert when unset.

VI. EVALUATION

A. Methodology

The workload is a 128 MiB file copied from the passed-through stick to the guest’s own disk with `dd`, in a `systemd` unit that starts at boot. The harness waits until the guest reports that it has copied 16 MiB—a fixed *fraction* of the file, read from the copy progress the guest puts in its heartbeat, rather than a fixed wall-clock lead—and then requests the migration. A wall-clock lead is not portable across builds: the release build copies the same file several times faster than the instrumented debug build, and a lead tuned for one lets the copy finish before the migration is even requested. A run is successful only if all four of the following hold: the copy window strictly contains

the *whole* migration, from the request to the completion of the switchover (both recorded by the harness); the MD5 of the copied file equals the MD5 of the source file on the stick; the guest shows no enumeration after the migration uptime; and no reset or I/O error appears. The MD5 comparison is made inside the guest against the stick itself, so it also verifies that the device remained readable after the migration. The migration was requested with CH’s `downtime_ms=300` as the per-iteration stop-and-copy goal; the separate project acceptance target, used for the verdict, was ≤ 2000 ms. All runs were executed consecutively on the same host with the same stick and the same 128 MiB file, whose content was generated once and never regenerated between runs.

A single arm cannot separate the effects that a migration experiment confounds, so the acceptance arm (20 runs of the fixed system, debug build) is accompanied by four further arms which differ in exactly one factor: 8 runs with no migration at all (does the workload alone produce the result?), 8 runs with the release build (is the debug build’s instrumentation doing the work?), 8 runs with the hand-over kick disabled (is the kick load-bearing?), and three runs of the naive alternative in which the device is detached and re-attached with the hotplug interface instead of migrating. Section VI-H adds a fault-injection suite that makes the hand-over race happen by construction, so that the fixes can be credited rather than merely not falsified.

B. Results

All 20 runs pass. Table III lists the per-run figures; Figure 4 shows the downtime distribution. Downtime is measured by CH (`downtime_ms`), not by the harness. Copy duration is the time the guest takes to copy the file, which is chosen to be long enough that the migration necessarily falls inside the copy window.

How much does 20 out of 20 establish? More than the ten-run version of this experiment did, but still not much on its own, and we prefer to say so. 20 consecutive successes give a one-sided 95 % lower confidence bound of 0.86 on the pass probability and a one-sided 95 % upper bound of 0.14 on the residual failure rate; the two-sided 95 % Clopper-Pearson interval is $[0.83, 1.00]$. Against the pre-fix behaviour we observed—one failure in five runs—the comparison is still not statistically significant (Fisher exact, $p = 0.20$ against the five-run arm). 20 successes bound the residual rate at roughly one in seven; they do not by themselves show *which* fix carries the weight. That is what the negative controls and the injection suite of Sections VI-C–VI-H are for, and Section VI-K discusses what would be needed to narrow the bound further.

What power do the comparisons have? We report this rather than leaving it implicit. Taking the observed rates as the truth, the natural kick comparison (7/8 against 20/20) has exact unconditional power 0.067 at the sample sizes we used; reaching 80 % power against the same effect would need about 62 runs per arm, which is why we did not try to settle the kick question with more natural runs and instead used the injection

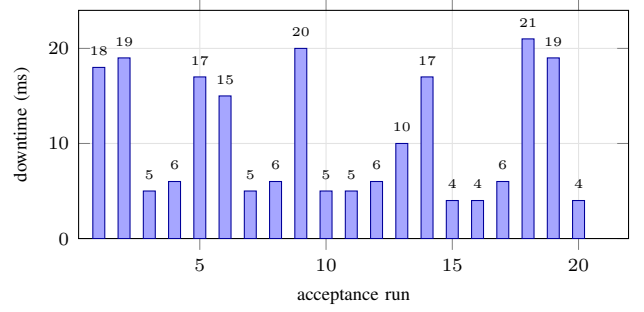


Fig. 4. Measured downtime of the 20 acceptance runs (source: `data/runs.dat`, from `results-debug.csv`). The goal of ≤ 2000 ms is far above the plotted range.

suite. The 5 s injection pair is discussed in Section VI-H, including the power it does and does not have.

Was the hand-over path actually exercised? Yes, in every run, and the harness records it. Each run shows two interrupt-line installations (the source registering at start-up, the destination taking over during the migration) and between 2 and 2 re-raised interrupts on the new line, and one stale teardown that the server refused (between 1 and 1 per run, logged as ignoring IRQ disable from stale `vfio-user client`). Without that refusal the destination would have been left with a dummy interrupt line, which is the failure mode we measured before the guard existed. The counts come from the server log of each run, which is archived with the raw data.

The downtime figures are also less precise than their range suggests: the median is 6.0 ms but the bootstrap 95 % interval for the median is $[5, 17]$ ms, against an observed range of 4–21 ms. The 20 values are not unimodal: eleven runs give 4–6 ms, eight give 15–21 ms and one gives 10 ms, and the groups alternate rather than drift (runs 1–2 give 18 and 19 ms, runs 3–4 give 5 and 6 ms, runs 5–6 give 17 and 15 ms). We cannot attribute this to the co-resident VMs of Section III-A as we first assumed: joining the archived host-load samples to each run’s migration instant gives a slightly *negative* correlation between the one-minute load average and downtime ($r = -0.38$ for the acceptance arm; the high-downtime runs average a lower load than the low-downtime ones), which points more to the switchover cost depending on the device and queue state than to host CPU. The one-minute load average is also too coarse to resolve a few-millisecond stop-and-copy phase, so we report the pattern and the failed hypothesis rather than a cause. It is anyway not a property of the USB path, which is idle during the switchover. Copy duration varied between 13.3 s and 32.7 s; the no-migration control (8 runs, median 26.7 s) shows the same spread, so we do not attribute that variation to the migration.

C. Controls and negative controls

Table IV puts the acceptance arm beside the four comparison arms. Two of them bound our claims from opposite sides.

TABLE III

ACCEPTANCE RUNS (POST-FIX; SOURCE: RESULTS-DEBUG.CSV, ONE ROW PER RUN). “SPANS” MEANS THE COPY WINDOW CONTAINS THE WHOLE MIGRATION; “ENUMS” COUNTS GUEST ENUMERATIONS AFTER THE MIGRATION UPTIME.

Run	Downtime (ms)	Copy (s)	Spans	MD5 / enums
1	18	26.4	yes	match / 0
2	19	32.7	yes	match / 0
3	5	26.6	yes	match / 0
4	6	13.4	yes	match / 0
5	17	22.9	yes	match / 0
6	15	31.5	yes	match / 0
7	5	28.9	yes	match / 0
8	6	13.7	yes	match / 0
9	20	21.6	yes	match / 0
10	5	31.9	yes	match / 0
11	5	28.8	yes	match / 0
12	6	13.4	yes	match / 0
13	10	22.6	yes	match / 0
14	17	31.9	yes	match / 0
15	4	28.7	yes	match / 0
16	4	13.3	yes	match / 0
17	6	17.3	yes	match / 0
18	21	25.7	yes	match / 0
19	19	26.7	yes	match / 0
20	4	25.6	yes	match / 0

No migration. The 8 control runs copy the same file on the same guest without migrating. All complete with matching checksums, so the workload by itself is sound and the copy-duration spread is not caused by the migration. Because there is no migration, these runs are judged on the digest alone: the span, enumeration and downtime criteria do not apply, and the arm cannot rule out a guest-visible disturbance that leaves the final digest intact. These runs deliberately have no downtime figure either.

Kick disabled. The 8 runs use the acceptance build with the hand-over interrupt suppressed by a debug-only environment switch. This is the arm that decides whether the kick earns its place: if these runs failed at the rate suggested by the exposure measurement, the kick would be clearly load-bearing; if they all passed, it would be defensive decoration. The outcome is in between—7/8 pass, one run stalls at a fixed byte count with no re-enumeration; its guest console shows no error until the block layer’s command timeout fires about 30 s later, and the copy never reports completion, which is the lost-completion signature—and on this sample alone the difference from the acceptance arm is not significant (Fisher exact, $p = 0.286$). The reason is structural rather than statistical: a missed completion is only fatal when it is the last one outstanding (Section VI-G), so an arm that leaves the window to chance can only fail in the fraction of runs where that coincidence occurs. Section VI-H therefore removes the luck by enlarging the window by construction.

Release build. The 8 release runs copy at a median of 5.8 s against 26.0 s for the debug build; the instrumentation (tracing at `-v`, packet capture) therefore slows the transfer noticeably but does not change the outcome, which is what matters for the argument. All 8/8 pass.

TABLE IV

COMPARISON ARMS (SOURCE: RESULTS-<TAG>.CSV OF EACH ARM). “PASS” COUNTS RUNS MEETING THE VERDICT OF SECTION VI FOR THE MIGRATING ARMS; THE NO-MIGRATION CONTROL HAS NO MIGRATION TO SPAN AND NO MIGRATION-UPTIME CUTOFF, SO IT IS JUDGED ON THE COPY DIGEST ALONE AND ITS 8/8 IS A CHECKSUM CHECK RATHER THAN THE FULL VERDICT. DOWNTIME IS ONLY DEFINED FOR THE MIGRATING ARMS.

Arm	Pass	Copy med. (s)	Downtime (ms)	Note
Debug, migration	20/20	26.0	4–21	—
Release, migration	8/8	5.8	6–21	—
Debug, no migration	8/8	26.7	n/a	n/a
Debug, kick disabled	7/8	21.8	4–18	$p = 0.286$ vs. debug

D. The naive alternative: detach and re-attach

The obvious way to move a passed-through device to another VM without changing the host-side server is to let the host-side session go and make a new one: detach the device, then attach it again. We measured what the guest sees in that case with the same workload and the same guest-log criteria, replacing the migration with a hotplug detach four seconds into the copy and a re-attach five seconds later.

In all 3 baseline runs the copy failed. The guest’s own kernel log records `usb 1-1: USB disconnect`, device number 2 at the detach moment, followed by up to 5 `blk_update_request: I/O error` lines for reads that were in flight on `sda`; `dd` returned non-zero in every one of the 3 runs and the digest of the partial file matched the source in 0 of them. No re-enumeration appears, because the copy had already aborted before the re-attach could bring the device back: the run is over from the guest’s point of view the moment the device disappears.

That is the cost the design avoids, and it is the same class of event as the one Section III attributes to tearing the session down—but here the guest sees it directly. It also shows why “detach and re-attach” is not an acceptable implementation of the same-host hand-over: it does not merely disturb the guest, it fails the transfer.

E. Timeline of a representative run

Figure 5 shows the guest’s copy progress for run debug-1 of the acceptance arm, taken from the copy size the guest reports in its own heartbeat. The curve is not perfectly smooth, but it continues across the migration and finishes with $rc = 0$; the guest never restarts the transfer. The mean throughput of this run (128 MiB in 26.4 s) is consistent with the stick’s USB 2.0 read performance measured outside the migration.

F. Data-path continuity under the hand-over

Figure 6 compares the USB packet rate across the hand-over before and after the interrupt fix, both taken from the `vfiio-user` server’s packet capture. Before the fix the rate drops to zero at the hand-over and remains near zero: the device is still powered and the server still holds it, but no further transfer is initiated because the guest is waiting for a completion it was never told about. After the fix the rate continues across the hand-over and only falls to zero when the copy has finished.

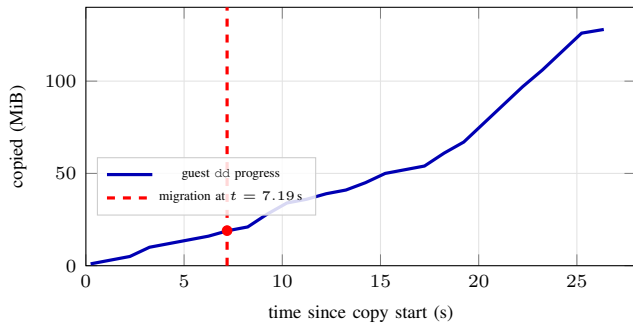


Fig. 5. Guest copy progress in acceptance run debug-1 (128 MiB; source: the `copied=` field of the guest’s own heartbeat). The migration is requested at $t = 7.19$ s, with 19 MiB copied, and the copy completes at $t = 26.4$ s with status 0: the transfer continues across the hand-over.

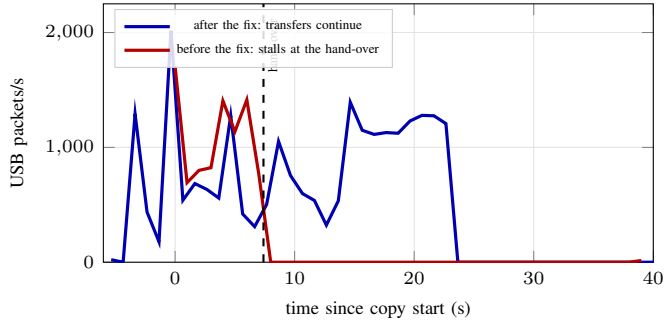


Fig. 6. USB packets per second across the hand-over, counted by `usbmon` at the `vfio-user` server. Both traces are single runs, so this is an illustration of the mechanism rather than a controlled A/B comparison. Without the interrupt fix the transfers stop at the hand-over and do not recover within the observed window; with the fix they continue until the copy completes.

This figure is the clearest evidence that the failures we saw were a lost-notification problem rather than a transport or device problem.

G. How exposed the runs were to the race

The hand-over fix only matters if the race it addresses actually occurs, so we measured its exposure instead of inferring it. Two things have to coincide: a transfer must complete while the source VMM will never service another interrupt, and before the interrupter has installed the destination’s line. The server log records every completion and every line installation with microsecond timestamps, and the VMM log records the instant at which it paused the source (which is a few milliseconds *after* the migration is requested, because CH pre-copies memory while the guest is still running). Anchoring the start of the window at the pause therefore measures the race; anchoring it at the request overstates it, because completions signalled before the pause are serviced normally. We report the pause-anchored count as the measurement and the request-anchored count as a strict upper bound.

The window was 3.0–17.7 ms wide, and 8 of the 20 acceptance runs had at least one completion inside it: 10 completions in total, at most 2 in a single run. For each of those completions

the event is in the guest’s event ring but the interrupt went to the departing VMM’s descriptor, so the guest was not told about it. The exposure rate is 40 %. Counting from the request instead would give 11 of 20 runs and 24 completions (55 %); we use the smaller number because it is the one the timestamps support, and the difference is itself informative about how much of the pre-copy window is still live.

Whether such a missed interrupt is fatal depends on whether it is the *last* outstanding completion. The stick is a USB 2.0 Bulk-Only-Transport device and the guest’s mass-storage path keeps only a few commands in flight, so if the completion of the final outstanding command is the one that lands in the window, no further transfer is submitted and no later completion arrives to interrupt the guest and drain the ring: the copy stops at a fixed byte count, which is the stall we reproduced. If, on the other hand, more commands were still queued, the next completion interrupts the guest on the new line and it drains the ring, including the events it was never interrupted for, and the same exposure is harmless. That is why the defect is intermittent in the acceptance runs rather than deterministic—and why the *kick-off* arm of Table IV is a weak test on its own: it can only fail in the fraction of runs where the missed completion happens to be the last one. The injection suite of Section VI-H makes that condition explicit.

H. Fault injection at the hand-over

The exposure measurement shows that the race happens on its own; it does not by itself show that the two hand-over fixes are what make the system survive it. To show that, we added two environment-gated hooks that exist only in debug builds: one sleeps inside the `SetIrqs` handler for a configured time *before* the destination’s line reaches the interrupter worker, and the other makes every connection believe it owns the device, reintroducing the stale-teardown defect. The first widens the hand-over window on demand; the second disables the ownership guard.

The placement of the delay is the point. While `SetIrqs` sleeps, the interrupter worker keeps draining completion events onto the departing client’s line—the window the kick exists to cover—and the transfer queue empties, so that the completion which lands in the window is the last one outstanding. The *number* of completions at risk does not grow with the delay: it saturates at the queue depth (0–4 naturally, 1–3 at 500 ms, 1–4 at 5 s), because once the queue is empty there is nothing left to complete. What the delay changes is whether a *further* completion follows the lost one. Sleeping in the worker instead would push events behind the line-swap message and *shrink* the exposure rather than enlarge it. Two caveats about the hook’s scope: it is not restricted to the hand-over, so it also delays the source’s own registration at start-up (visible in the server log as a 500 ms or 5 s “TEST HOOK” line at boot), and the sleep happens while the shared backend lock is held. Both are irrelevant to the A/B contrast, which compares the same binary with the same hook enabled and only the kick switched, but they are part of what the arm measures. Table V gives the outcome; the VMM-side effect of the delay is directly visible

TABLE V

FAULT-INJECTION ARMS (DEBUG BUILD; SOURCE: RESULTS-*.CSV FROM GUEST/INJECTION-SUITE.SH). “EXPECTED” IS WHAT THE MECHANISM PREDICTS; THE LAST COLUMN IS OBSERVED. THE HOOKS ARE INERT WHEN UNSET, SO ALL ROWS RUN THE SAME BINARY. THESE ARE MECHANISM TESTS, NOT ACCEPTANCE RUNS.

Injection	Expected	Observed (pass)
Hooks dormant	pass	5/5
500ms window, kick on	pass	5/5
500ms window, kick off	fail	4/10
5s window, kick on	pass	8/8
5s window, kick off	fail	2/8
Owner guard off	fail	0/5

as downtime (the 5s arms report ≈ 5 s, and they are judged with the downtime budget raised to 12s precisely because the injected delay *is* the downtime there).

Three things follow. First, the hooks do nothing when they are not enabled (5/5), so the differences below are caused by the injections and not by the build. Second, the *ownership guard is load-bearing and its absence is deterministically fatal*: with the guard disabled the copy fails in all 5 runs (0/5 pass; Fisher exact against the acceptance arm, $p = < 0.0001$), `dd` returns non-zero and the guest kernel logs a 36s `DID_TIME_OUT` followed by repeated I/O errors—the source VMM’s teardown has replaced the destination’s interrupt line with a dummy, so the guest’s commands never complete normally.

Third, the kick is what makes the hand-over survive a window that is wide enough to matter. With a 500ms window the kick-enabled arm passes 5/5 and the kick-disabled arm passes 4/10 ($p = 0.044$). With a 5s window the contrast is sharper still: 8/8 against 2/8 ($p = 0.0070$). In every failing run the copy stops at a fixed byte count and the guest sends no further completion until a SCSI timeout expires, i.e. the observed failure is the mechanism the kick prevents, not a generic timeout. Note that the kick-off arms are *not deterministically fatal* even at 5s—two of eight runs recovered—which is consistent with the account in Section VI-G: a wide window drains the queue and makes it likely, but not certain, that the lost completion is the last one outstanding.

Two honesty notes on how the 5s pair came about. First, it is a *post hoc* extension: the suite was designed with four arms and predicted that the 500ms window without the kick would fail. At the original five runs that prediction looked wrong (4/5 passed); after extending the arm to ten runs the effect is clear (4/10), which is itself an illustration of why small arms mislead. We added the 5s pair only after that, extended it to a fixed eight runs per arm rather than stopping when it looked good, and report it as exploratory rather than confirmatory. At eight runs per arm, with a deliberately imperfect effect (5% failure with the kick, 80% without) the arm has power 0.88; 8 runs per arm would be needed for 80% power against that effect, so the arm is adequately powered for a large effect and not for a small one. Second, the ownership hook disables both halves of the guard, including the DMA-unmap half that our

TABLE VI
DEFECTS FOUND AND FIXED.

Defect	Symptom	Fix
single-accept server	destination blocks 199.9s in the handshake	per-command lock, multi-client accept
stale teardown	guest I/O times out and the copy fails after migration	interrupt-ownership rule
lost hand-over interrupt	copy stalls with no error (intermittent)	re-raise one interrupt on registration
inverted reset-capability parse (crate)	every connection issues a device reset	equality test instead of inequality

workload does not otherwise exercise, so the arm demonstrates the guard as a whole rather than the two halves separately.

One caveat is worth stating: the injection arms are mechanism tests, not acceptance runs. They hold the workload fixed and change only the server’s behaviour through a dormant hook, but the enlarged window is synthetic, so the arms establish that the fixes are necessary and sufficient *for this mechanism*; they do not measure how often the mechanism decides the outcome in production.

I. Defects found

Table VI lists the four things that had to be fixed: the single-accept limitation itself and the three defects it exposed. Three are deterministic: the single-accept server blocks the destination in the handshake on every run, the inverted reset-capability parse makes every connection reset the device, and—as the injection of Section VI-H shows—disabling the ownership guard makes the copy fail in every run. Only the lost hand-over interrupt is timing-dependent, and it is the one that had to be demonstrated by enlarging the window rather than by repeating natural runs.

J. Discussion of the measurements

Two points about the method are worth stating explicitly, because both changed our conclusions during the work. First, host-side observation is not sufficient: the packet capture and the VMM logs looked healthy in a run whose guest had in fact stopped making progress. Second, a single passing run is not evidence for a timing-dependent fix: the code without the fix of Section IV-D passed four consecutive runs before the defect appeared on the fifth, and only repeated runs after the fix justify the claim of 20 out of 20.

K. Threats to validity

Construct. The integrity check is an MD5 of a read-only bulk copy that was read once, sequentially. It detects corrupted or lost data, but it cannot detect *duplicated* reads: a replayed read changes nothing in the destination file. Our claim is therefore *no data corruption and no lost I/O*, and we do not claim to have tested duplication. The enumeration and reset criteria are now derived from the guest’s own kernel log and

are counted relative to the guest’s uptime at the migration instant (Section V), which removes an earlier threshold artefact in our harness, but they still rely on the kernel printing a message. It should also be said that in this dataset they never decided a verdict on their own: every failing run already failed the span or digest criterion, and the stalled runs have no dmesg section at all, which the verdict treats as a failure rather than as “nothing happened”. The independent discriminating power of the two kernel-log criteria is therefore demonstrated by construction (they fail closed), not by a run they happened to catch.

Internal. The host was loaded by four other VMs; a host-load sample is archived with each batch, but it is not synchronised to individual runs. Run order was not randomised and the arms were executed in blocks (acceptance, control, release, kick-off), so a slow drift in host conditions would be confounded with the arm—this is the single largest threat to the arm comparison of Table IV. The injection suite of Section VI-H is designed to be immune to it: it forces the hand-over condition inside a single run and compares presence and absence of the fix within one build, so its conclusions do not depend on cross-arm timing. The server was a debug build with verbose logging and packet capture enabled, which is why the release arm exists. The migration and pause instants used for the “spans” test are host-clock times; the guest’s copy timestamps are compared against them assuming the two clocks agree, and that assumption is not verified: the heartbeat pairs check that the guest clock runs at the same rate, not that its offset is zero. The exposure in the data is small—the nearest enumeration or reset event is at least 6.9 s from the cutoff in every migrating run, and the span margin is about 7 s—but the offset itself is untested. The destination of the copy was a file on the host-backed guest disk, so host page-cache writeback is in the loop.

External. One device class (bulk storage), one USB 2.0 Bulk-Only Transport device on one controller, one guest kernel, one workload size, and same-host `memfd`s migration only. A USB 3.0/UAS device with several in-flight commands would exercise the hand-over differently, and the slow transfer of the stick used here (128 MiB in 13.3–32.7 s, roughly 4–10 MiB/s) makes a completion less likely to land inside the hand-over window than it would be on faster hardware.

Statistical. $n = 20$ in the acceptance arm and $n = 5$ for the pre-fix rate, so the confidence intervals are wide. The natural kick comparison has power 0.067 and would need about 62 runs per arm to reach 80 % at the observed effect; the 5 s injection pair reaches $p = 0.0070$ at 8/arm but has power 0.88 against an imperfect effect. Section VI-B and Section VI-H give the full statement, including that the 5 s pair was added after the 500 ms arm contradicted its prediction and is therefore exploratory. The pre-fix five-run series is described in the development log; its raw logs predate the archiving harness and are not part of the attached evidence, so the pre-fix comparison should be read as qualitative.

VII. DISCUSSION AND LIMITATIONS

a) What is transparent.: At the USB level the guest is undisturbed: it enumerates the device once, at boot, and never again; it observes no reset, no port status change and no I/O error; the file it copies is byte-identical to the source. This is the property that matters for storage passthrough and it is the one we set out to obtain.

b) What is not transparent.: One guest-visible disturbance remains and we do not want to hide it: after the migration the guest reprints its login banner. The destination re-creates the serial device, which resets the guest TTY. This is a property of the VMM’s serial device model, not of the USB path, and it does not affect the device. We report it because a claim of “zero perception” should be stated at the granularity at which it is true.

c) No explicit quiesce.: We did not implement a bounded drain of in-flight transfers before the hand-over. Transfers in flight when the migration happens are handled by the re-raise mechanism: the injected 5 s-window experiment (Section VI-H) shows that without that one re-raised interrupt the guest can wait indefinitely for a completion that has already been written to its event ring, and that with it the copy continues even when the window is three orders of magnitude wider than the natural one. What we cannot claim is that a transfer is never retried: an explicit quiesce would make the in-flight set empty by construction and remove the dependence on where in the command queue the hand-over lands.

d) Scope.: The evaluation covers bulk storage on a USB 2.0 stick, one device per controller, Linux guests, and same-host migration. Other classes (HID, isochronous audio, serial) have different in-flight semantics; isochronous traffic in particular cannot be made lossless by retrying and is not supported by the underlying `nusb` library. Cross-host migration is a different problem: it needs the device state to be transferred or an equivalent device to be provided on the destination, and it cannot preserve a host-side session.

e) Failure and rollback.: Ownership of the device is decided by whichever connection most recently registered interrupts, and it is not committed to a migration transaction. If a migration is cancelled after the destination has installed its line, the source is left running with a line that will not be serviced, and nothing re-registers it. Our own Phase-0 measurements showed that CH’s `timeout_strategy=cancel` did not in fact bring the source back to a usable state. A production design needs an explicit epoch that is committed at migration commit time, plus a revoke and re-register path, and it needs to be tested with a destination crash and a source-side cancel while I/O is in flight. We did not implement or test that.

f) Socket trust and bounds.: The `vfi`o-user socket grants whoever can connect it full DMA authority over the guest’s memory, so it is as sensitive as the guest’s own memory. Our accept loop bounds how many connections are *served* (`--max-clients`), but a client beyond that limit waits in the listen backlog, there is no per-client timeout, and a client that registers interrupts becomes the owner and may then unmap

DMA regions. A deployment must therefore treat the socket as trusted and protect it with filesystem permissions or a private namespace; the design does not defend against a hostile local client.

g) *Process lifecycle.*: With more than one client the server no longer exits when the last client disconnects, which is a deliberate change needed for the hand-over. Deployments that rely on exit-on-disconnect (for example socket-activated systemd units) must either keep the default single-client mode or manage the process explicitly.

VIII. CONCLUSION AND FUTURE WORK

We set out to make a USB storage device survive a live migration of the VM it is passed through to, and we found that the path did not merely fail—it deadlocked, because a vfio-user server built for one client cannot serve the destination while the source is still attached. The fix follows from a property of the protocol that is easy to miss: the handshake does not touch the device, so the device lock does not have to be held for the lifetime of a connection. On top of that, two further defects had to be fixed before the hand-over was correct rather than merely possible: ownership of destructive teardown, and the synchronisation of the interrupt line across the hand-over. With those in place, a 128 MiB copy that spans the migration completes in 20 out of 20 runs with 4–21 ms of downtime and byte-identical data; the kick-disabled and guard-disabled injections, and the enlarged-window injections, show that both hand-over fixes are load-bearing rather than incidental.

Future work falls into three groups. First, an explicit quiesce and a guest-cooperative protocol would make in-flight behaviour deterministic instead of relying on retry. Second, a protocol-level device-state region would allow the device to be reconstructed on another host, which is the prerequisite for cross-host migration; our measurements show that the current protocol carries none of the required state. Third, the same technique should be evaluated for other device classes and for multiple devices on one controller, where the ownership rules we introduced would need to become per-device.

APPENDIX A REPRODUCTION

The full harness, the development log and the raw measurement data accompany this submission as an artefact (the guest harness under `guest/`, the paper and its figures under `paper/`, the run logs and driver scripts under `artifacts/` and the development log under `docs/DEVLOG_cn.md`). The repository location is omitted here for double-blind review. In outline:

- 1) build the guest image: `guest/build-guest.sh`;
- 2) place a 128 MiB `testfile.bin` on the stick and record its MD5 in `guest/testfile.md5`;
- 3) run `guest/usb-migration-demo.sh` (add `PAUSE=1` for a step-by-step narrated run);
- 4) read the verdict, which is derived from the guest’s own log as described in Section V;

- 5) reproduce the whole measurement set with `guest/campaign-b.sh` all (acceptance, controls and the naive baseline) and `guest/injection-suite.sh` (the fault-injection arms);
- 6) archive each batch with `guest/collect-artifacts.sh`, which writes per-file SHA-256 checksums and a manifest next to the raw logs.

REFERENCES

- [1] C. Clark, K. Fraser, S. Hand, J. G. Hansen, E. Jul, C. Limpach, I. Pratt, and A. Warfield, “Live migration of virtual machines,” in *Proc. 2nd USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2005, accessed: 2026-09-20. [Online]. Available: <https://www.usenix.org/conference/nsdi-05/live-migration-virtual-machines>
- [2] Linux kernel, “VFIO device migration interface (include/uapi/linux/vfio.h),” kernel user-space API header; the migration commands and feature bits are defined there. Accessed: 2026-09-20. [Online]. Available: <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/include/uapi/linux/vfio.h>
- [3] Cloud Hypervisor project, “VFIO-user HOWTO (docs/vfio-user.md),” accessed: 2026-09-20. [Online]. Available: <https://github.com/cloud-hypervisor/cloud-hypervisor/blob/main/docs/vfio-user.md>
- [4] Cyberus Technology GmbH, “usbvfiod: A vfio-user server for USB pass-through,” accessed: 2026-09-20. [Online]. Available: <https://github.com/cyberus-technology/usbvfiod/tree/4d2c5af>
- [5] K. Mehall, “nusb: A new pure-rust library for cross-platform low-level USB,” accessed: 2026-09-20. [Online]. Available: <https://github.com/kevinmehall/nusb>
- [6] Cloud Hypervisor project, “Live migration (docs/live_migration.md),” accessed: 2026-09-20. [Online]. Available: https://github.com/cloud-hypervisor/cloud-hypervisor/blob/main/docs/live_migration.md
- [7] A. Kivity, Y. Kamay, D. Laor, U. Lublin, and A. Liguori, “kvm: the linux virtual machine monitor,” in *Proc. Linux Symposium*, vol. 1, 2007, pp. 225–230, accessed: 2026-09-20. [Online]. Available: <https://www.kernel.org/doc/ols/2007/ols2007v1-pages-225-230.pdf>
- [8] A. Agache, M. Brooker, A. Florescu, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa, “Firecracker: Lightweight virtualization for serverless applications,” in *Proc. 17th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2020, accessed: 2026-09-20. [Online]. Available: <https://www.usenix.org/conference/nsdi20/presentation/agache>
- [9] Cloud Hypervisor project, “Cloud hypervisor: A virtual machine monitor for modern cloud workloads,” accessed: 2026-09-20. [Online]. Available: <https://github.com/cloud-hypervisor/cloud-hypervisor>
- [10] Linux man-pages project, “memfd_create(2),” accessed: 2026-09-20. [Online]. Available: https://man7.org/linux/man-pages/man2/memfd_create.2.html
- [11] The Linux Kernel documentation, “VFIO — virtual function I/O,” accessed: 2026-09-20. [Online]. Available: <https://docs.kernel.org/driver-api/vfio.html>
- [12] QEMU project, “vfio-user protocol specification,” accessed: 2026-09-20. [Online]. Available: <https://www.qemu.org/docs/master/interop/vfio-user.html>
- [13] Nutanix, “libvfio-user: Framework for emulating devices in userspace,” accessed: 2026-09-20. [Online]. Available: <https://github.com/nutanix/libvfio-user>
- [14] The vfio_user authors, “vfio_user 0.1.5 — rust crate documentation,” accessed: 2026-09-20. [Online]. Available: https://docs.rs/vfio_user/0.1.5
- [15] rust-vmm project, “rust-vmm/vfio,” accessed: 2026-09-20. [Online]. Available: <https://github.com/rust-vmm/vfio>
- [16] H. Komatsu, “[PATCH v2 0/8] vfio-user: Live migration support,” QEMU development mailing list, September 2026. Accessed: 2026-09-20. [Online]. Available: <https://lists.gnu.org/archive/html/qemu-devel/2026-09/msg03538.html>
- [17] H. Watanabe, H. Masaoka, T. Ohigashi, T. Kondo, K. Nishimura, and R. Aibara, “Supporting USB devices for the global migration,” in *Proc. 10th Annu. Int. Symp. Applications and the Internet (SAINT)*. IEEE Computer Society, 2010, pp. 153–156.

- [18] T. Hirofuchi, E. Kawai, K. Fujikawa, and H. Sunahara, "USB/IP: A transparent device sharing technology over IP network," *IPSJ Digital Courier*, vol. 1, 2005, accessed: 2026-09-20. [Online]. Available: https://www.jstage.jst.go.jp/article/ipsjdc/1/0/1_0_394/_article
- [19] The Linux Kernel documentation, "USB/IP protocol," accessed: 2026-09-20. [Online]. Available: https://docs.kernel.org/usb/usbip_protocol.html
- [20] QEMU project, "USB emulation," accessed: 2026-09-20. [Online]. Available: <https://www.qemu.org/docs/master/system/devices/usb.html>
- [21] libvirt project, "libvirt source: `qemuMigrationSrcIsAllowedHostdev`," permits migration of a domain with a USB host device and forbids other host devices; the source comment calls the USB case "migrated" in quotation marks, i.e. the device is detached and re-attached. Accessed: 2026-09-20. [Online]. Available: https://gitlab.com/libvirt/libvirt/-/blob/master/src/qemu/qemu_migration.c
- [22] SPICE project, "usbredir: USB device redirection protocol," accessed: 2026-09-20. [Online]. Available: <https://www.spice-space.org/usbredir.html>
- [23] Intel Corporation, "extensible host controller interface for universal serial bus (xHCI) specification, revision 1.2," accessed: 2026-09-20. [Online]. Available: <https://www.intel.com/content/dam/www/public/us/en/documents/technical-specifications/extensible-host-controller-interface-usb-xhci.pdf>
- [24] USB Implementers Forum, "Universal serial bus mass storage class bulk-only transport, revision 1.0," 1999, accessed: 2026-09-20. [Online]. Available: https://www.usb.org/sites/default/files/usbmassbulk_10.pdf